Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory work 3: Study and Empirical Analysis of Graph Traversal Algorithms (BFS and DFS)

Elaborated:
st. gr. FAF-241

Tacu Loredana

Verified:
asist. univ.

Fiștic Cristofor

Chișinău – 2026

1 Algorithm Analysis

1.1 Objective

The objective of this laboratory work is to perform an empirical analysis of graph traversal algorithms by measuring and comparing their execution times across different graph structures and sizes. The algorithms under study are:

- **Breadth-First Search (BFS)** — iterative, queue-based, level-by-level traversal
- **Depth-First Search — Iterative ($\mathbf{DFS_{iter}}$)** — iterative, explicit stack-based traversal
- **Depth-First Search — Recursive ($\mathbf{DFS_{rec}}$)** — recursive traversal (tested on small graphs only)

The goal is to validate the theoretical time complexity $O(V + E)$ through experimental measurements and to observe how different graph topologies (sparse, dense, tree, path, cycle, grid) affect the practical performance of each algorithm.

1.2 Tasks

1. Implement BFS (iterative), DFS (iterative), and DFS (recursive) in Python with support for disconnected graphs.
2. Generate six families of test graphs: sparse random, dense random, random tree, path, cycle, and 2D grid.
3. Test each algorithm on node counts $n \in \{100, 500, 1000, 2000, 5000, 10000\}$.
4. Measure execution time using `time.perf_counter_ns()` and average over 5 independent runs.
5. Export results to CSV and produce comparison charts.
6. Create an animated GIF visualisation demonstrating BFS and DFS traversal on a 20-node demo graph.
7. Compare empirical results with theoretical complexity predictions.

1.3 Theoretical Notes

1.3.1 Introduction

Graph traversal is the process of visiting every reachable vertex in a graph exactly once. It is a foundational operation in computer science, underpinning algorithms for shortest paths, connectivity, cycle detection, topological sorting, and more.

The two classical traversal strategies differ in how they prioritise which vertex to expand next:

- **BFS** uses a FIFO queue: it fully explores all vertices at depth d before moving to depth $d+1$. This produces level-order traversal and guarantees shortest-path distances in unweighted graphs.
- **DFS** uses a LIFO stack (explicit or the call stack): it dives as deep as possible along one branch before backtracking. This is natural for cycle detection, topological sorting, and strongly connected component algorithms.

Both algorithms share the same asymptotic complexity in terms of the number of vertices V and edges E.

1.3.2 Complexity Summary

Table 1: Time and space complexity of the implemented graph traversal algorithms.

Algorithm	**Best**	**Average**	**Worst**	**Space**
BFS (iterative)	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$
DFS (iterative)	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$
DFS (recursive)	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$

All three algorithms are linear in the size of the graph. The practical differences arise from constant factors and memory access patterns, not from asymptotic complexity.

1.3.3 Comparison Metric

The primary metric is **wall-clock execution time**, measured in nanoseconds via `time.perf_counter_n` and reported in milliseconds. Each test is repeated 5 times and the arithmetic mean is taken:

$$\bar{T}_A(n) = \frac{1}{k}\sum_{i=1}^{k} T_A^{(i)}(n), \quad k = 5$$

A secondary metric is **nodes visited**, which verifies that all algorithms produce a complete traversal (`nodes_visited = V`).

1.3.4 Input Format — Graph Types

- **SPARSE_RANDOM** — Erdős–Rényi $G(n,p)$ with $p = \frac{4}{n-1}$; average degree ≈ 4, total edges $\approx 2n$.
- **DENSE_RANDOM** — Erdős–Rényi $G(n,p)$ with $p = 0.15$; total edges $\approx 0.075 \cdot n^2$.
- **TREE** — Random spanning tree: node i connects to a uniformly chosen parent in $[0, i-1]$. Exactly $n-1$ edges.
- **PATH** — Linear chain $0 \to 1 \to \cdots \to (n-1)$. Exactly $n-1$ edges; diameter $n-1$.

- **CYCLE** — Path with one additional edge $(n-1) \to 0$. Exactly n edges.
- **GRID** — 2D grid of side $\lfloor \sqrt{n} \rfloor$; actual node count $= \lfloor \sqrt{n} \rfloor^2$. Each node has degree 2, 3, or 4. Total edges $\approx 2n$.

Sizes tested: $n \in \{100, 500, 1000, 2000, 5000, 10000\}$. Recursive DFS is only tested for $n \leq 500$ to avoid exceeding Python's default call-stack limit.

2 Implementation

2.1 Breadth-First Search (BFS)

BFS explores a graph layer by layer using a FIFO queue. Starting from a source vertex, it visits all vertices at distance 1, then distance 2, and so on.

Algorithm:

1. Enqueue the start vertex and mark it as visited.
2. While the queue is non-empty:
   (a) Dequeue a vertex u.
   (b) For each unvisited neighbour v of u, mark v visited and enqueue it.
3. Repeat from any unvisited vertex to handle disconnected graphs.

Key characteristics:

- **Level-order traversal**: vertices are visited in non-decreasing order of their distance from the source.
- **Shortest-path property**: the first time BFS reaches a vertex, it has found the shortest hop-count path in an unweighted graph.
- **Queue memory**: at peak, the queue may hold all vertices in one frontier level, requiring $O(V)$ memory in the worst case.
- It is well-suited to dense graphs where the frontier stays small relative to the total number of processed edges.

```python
def bfs(graph, start=0):
    visited = set()
    order = []

    def _bfs_component(src):
        queue = collections.deque([src])
        visited.add(src)
        while queue:
            node = queue.popleft()
            order.append(node)
            for nb in graph[node]:
                if nb not in visited:
                    visited.add(nb)
                    queue.append(nb)

    _bfs_component(start)
    for node in graph:  # handle disconnected components
        if node not in visited:
            _bfs_component(node)
    return order, len(order)
```

Listing 1: BFS implementation

Complexity: $O(V + E)$. Every vertex is enqueued and dequeued once, and every edge is inspected at most twice because the graph is undirected.

2.2 Depth-First Search — Iterative

The iterative DFS replaces the call stack of the recursive variant with an explicit LIFO stack, avoiding Python's recursion limit and making it applicable to arbitrarily large graphs.

Algorithm:

1. Push the start vertex onto the stack.
2. While the stack is non-empty:
   (a) Pop a vertex u.
   (b) If u has already been visited, skip it.
   (c) Mark u as visited and push its unvisited neighbours in reverse order onto the stack.
3. Repeat from any unvisited vertex for disconnected graphs.

Key characteristics:

- **Deep traversal**: explores one path to its full depth before backtracking.
- **No recursion limit**: safe for graphs with thousands of nodes.

- The explicit stack may temporarily hold more entries than the implicit call stack of the recursive version because all neighbours may be pushed at once.
- It is especially efficient on sparse graphs such as trees, paths, and grids.

```python
def dfs_iterative(graph, start=0):
    visited = set()
    order = []

    def _dfs_component(src):
        stack = [src]
        while stack:
            node = stack.pop()
            if node in visited:
                continue
            visited.add(node)
            order.append(node)
            for nb in reversed(graph[node]):
                if nb not in visited:
                    stack.append(nb)

    _dfs_component(start)
    for node in graph:
        if node not in visited:
            _dfs_component(node)
    return order, len(order)
```

Listing 2: Iterative DFS implementation

Complexity: $O(V + E)$. Each vertex is processed once and each edge is inspected at most twice.

2.3 Depth-First Search — Recursive

The recursive DFS is the canonical textbook implementation. Python's call stack acts as the implicit DFS stack.

Algorithm:

1. Mark the current vertex as visited.
2. For each unvisited neighbour, recurse.
3. After recursion terminates, the vertex is fully explored.

```python
def dfs_recursive(graph, start=0):
    visited = set()
    order = []

    def _dfs(node):
        visited.add(node)
        order.append(node)
        for nb in graph[node]:
            if nb not in visited:
                _dfs(nb)

    _dfs(start)
    for node in graph:
        if node not in visited:
            _dfs(node)
    return order, len(order)
```

Listing 3: Recursive DFS implementation

Key characteristics:

- Simple and readable; directly mirrors the mathematical definition.
- **Stack overflow risk**: Python's default recursion limit is 1000. On a PATH graph of 1001 nodes, the recursion depth becomes n, causing a `RecursionError`. In this study, recursive DFS is restricted to $n \leq 500$.
- Slightly lower overhead than the iterative version on small graphs because no auxiliary stack object is needed.

Complexity: $O(V + E)$, with an $O(V)$ space cost from the call stack.

2.4 GIF Visualisation

In addition to the performance benchmarks, an animated GIF was produced for each algorithm on a hand-crafted 20-node graph. The graph has a tree skeleton with two cross-edges, making the difference between BFS (level-by-level) and DFS (depth-first) clearly visible.

Each frame of the animation colour-codes vertices as follows:

- **Light grey** — not yet discovered
- **Gold** — in the queue (BFS) or on the stack (DFS), waiting to be processed
- **Orange-red** — the vertex currently being expanded
- **Green** — fully visited

The animation is generated with `matplotlib.animation.FuncAnimation` and saved as a GIF via the `Pillow` writer at 2 frames per second.

2.5 Benchmarking Method

```python
def benchmark_single(algo_func, graph):
    total_ns = 0
    for _ in range(RUNS):
        start = time.perf_counter_ns()
        _, nv = algo_func(graph)
        end = time.perf_counter_ns()
        total_ns += (end - start)
    return total_ns // RUNS, nv
```

Listing 4: Benchmarking function

The same graph object is reused across all 5 runs for a given $(\text{algorithm}, \text{graph_type}, n)$ triple. Unlike sorting, which modifies the input in place, graph traversal is read-only, so no copy is needed. The graph is regenerated once per $(\text{graph_type}, n)$ pair, and all three algorithms are then benchmarked on the same instance.

3 Empirical Results

3.1 Effect of Graph Density

The most striking result is the divergence between sparse and dense graphs. On **SPARSE_RANDOM** graphs ($|E| \approx 2n$), both BFS and DFS_{iter} grow near-linearly: at $n = 10000$, BFS takes 2.58 ms and DFS takes 3.69 ms. On **DENSE_RANDOM** graphs ($|E| \approx 0.075n^2$), the growth is quadratic in n, consistent with $O(V + E) = O(n^2)$: BFS reaches 328.6 ms and DFS_{iter} reaches 1200.8 ms at $n = 10000$.

This confirms that the running time is dominated by the number of edges, not by the number of vertices alone.

3.2 BFS vs. DFS — Constant-Factor Difference

Across all graph types and sizes, **BFS is consistently faster than DFS_{iter}** by a factor of roughly 1.3× to 3.6×:

- On sparse, tree, path, cycle, and grid graphs, the ratio is approximately 1.3× to 1.5×.
- On dense graphs, the gap widens significantly, reaching approximately 3.6× at $n = 10000$.

The reason for the BFS advantage lies in data structure access patterns. BFS uses `collections.deque`, which provides amortised $O(1)$ `popleft()`. DFS_{iter} uses a Python list as a stack; although `pop()` from the end is $O(1)$, the list can grow large on dense graphs because neighbours are pushed eagerly, increasing memory pressure and cache misses.

3.3 Recursive vs. Iterative DFS

On the graphs where both were tested ($n \leq 500$), DFS_{rec} is slightly faster than or comparable to DFS_{iter}:

- At $n = 500$, SPARSE_RANDOM: $\text{DFS}_{\text{rec}} = 0.134$ ms, $\text{DFS}_{\text{iter}} = 0.173$ ms.
- At $n = 500$, DENSE_RANDOM: $\text{DFS}_{\text{rec}} = 0.717$ ms, $\text{DFS}_{\text{iter}} = 1.769$ ms.

The recursive version avoids the overhead of explicit stack management and instead relies on Python's native call-frame mechanism, which can be slightly cheaper on small graphs.

3.4 Graph Topology Effects

All sparse topologies (TREE, PATH, CYCLE, GRID) produce very similar timing curves because they all have $|E| = \Theta(n)$:

- At $n = 10000$, BFS ranges from 1.80 ms (PATH) to 2.59 ms (SPARSE_RANDOM), and DFS_{iter} ranges from 2.35 ms (PATH) to 3.69 ms (SPARSE_RANDOM).
- PATH and CYCLE graphs are the fastest because their adjacency lists have exactly two entries per node, minimising inner-loop work.
- TREE graphs show slightly higher variance; one run at $n = 100$ measured 0.214 ms for BFS due to OS scheduling noise.

3.5 Empirical Data Table

Selected results for BFS and DFS_{iter} on **SPARSE_RANDOM** and **DENSE_RANDOM**:

Table 2: Average execution time (ms) for BFS and DFS_{iter} on random graphs, averaged over 5 runs.

n	**Graph Type**	**BFS (ms)**	**DFS_{iter} (ms)**
100	SPARSE_RANDOM	0.020	0.031
500	SPARSE_RANDOM	0.113	0.173
1000	SPARSE_RANDOM	0.239	0.368
2000	SPARSE_RANDOM	0.650	0.748
5000	SPARSE_RANDOM	1.353	1.990
10000	SPARSE_RANDOM	2.582	3.694
100	DENSE_RANDOM	0.034	0.079
500	DENSE_RANDOM	0.692	1.769
1000	DENSE_RANDOM	2.817	7.133
2000	DENSE_RANDOM	11.424	29.428
5000	DENSE_RANDOM	76.698	188.183
10000	DENSE_RANDOM	328.597	1200.757

4 Conclusion

In this laboratory, Breadth-First Search and two variants of Depth-First Search were implemented in Python and empirically evaluated across six graph families and six node-count levels.

The experimental results confirm the theoretical analysis:

- **Both BFS and DFS run in $O(V+E)$ time.** On sparse graphs where $|E| = \Theta(V)$, the measured time grows linearly with n. On dense graphs where $|E| = \Theta(V^2)$, the measured time grows quadratically with n, exactly as $O(V + E) = O(V^2)$ predicts.
- **BFS is faster than $\mathbf{DFS_{iter}}$ in practice** by a constant factor across all tested configurations. The main reason is the lower overhead of `collections.deque` compared to a Python list used as a stack, particularly when the graph is dense and the stack grows large.
- **$\mathbf{DFS_{rec}}$ is slightly faster than $\mathbf{DFS_{iter}}$ on small graphs**, because it avoids explicit stack management. However, it is unsafe for large graphs due to Python's recursion depth limit, making $\mathrm{DFS_{iter}}$ the only practical DFS choice for $n \gtrsim 500$.
- **Graph topology matters for constant factors, not asymptotic complexity.** PATH and CYCLE graphs are fastest, while DENSE_RANDOM graphs are slowest. Trees, sparse random graphs, and grids all perform similarly.
- **Edge count is the dominant cost driver.** A 10 000-node dense graph with approximately 7.5 million edges is more than 100 times slower than a 10 000-node sparse graph with approximately 20 000 edges, despite the same vertex count.

Key takeaways:

1. When the graph is guaranteed to be sparse, both BFS and DFS remain practical even for very large vertex counts.
2. BFS should be preferred when shortest-path distances are needed in unweighted graphs.
3. Iterative DFS should be preferred when deep traversal is required and graph size may exceed Python's recursion limit.
4. Recursive DFS should be avoided for graphs with $n > 500$ unless the recursion limit is raised and the maximum depth is known to be safe.
5. For dense graphs, constant-factor optimisations may help, but the general traversal complexity remains $O(V + E)$.

Overall, the empirical measurements align closely with the theoretical predictions, reinforcing the importance of considering both vertex count and edge count when analysing graph algorithms in practice.